

**МИНОБРНАУКИ РОССИИ
ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ**

**ОТЧЁТ
ПО УЧЕБНОЙ ОЗНАКОМИТЕЛЬНОЙ ПРАКТИКЕ**

Визуализатор алгоритмов на графах

Студент гр. БИН-24-3

Руководитель, канд. техн. наук

Консультант, ассистент

_____ Н.М. Малышев

_____ Е.Ю. Соболевская

_____ М.В. Водяницкий

Владивосток 2026

МИНОБРНАУКИ РОССИИ
ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

на учебную ознакомительную практику

Студенту: гр. БИН-24-3 Малышеву Никите Максимовичу

Срок сдачи работы: 16.07.2026

Задание 1. Анализ поставленной задачи. Изучение теории графов и алгоритмов обхода в ширину (BFS) и в глубину (DFS), алгоритма Дейкстры для поиска кратчайших путей, алгоритмов Прима и Краскала для построения минимального остовного дерева.

Задание 2. Сбор и анализ информации. Обзор существующих визуализаторов алгоритмов на графах, сравнение технологий реализации, обоснование выбора стека разработки.

Задание 3. Разработка интерактивного веб-приложения – визуализатора алгоритмов на графах с модульной архитектурой: реализация редактора графа, пошаговой визуализации алгоритмов BFS, DFS, Дейкстры, Прима и Краскала с подсветкой рёбер и вершин, выводом порядка обхода и суммарного веса.

Задание 4. Оформить отчёт по практике в соответствии с СК-СТО-ТР-04-1.005-2015 «Требования к оформлению текстовой части выпускных квалификационных работ, курсовых работ (проектов), рефератов, контрольных работ, отчётов по практикам, лабораторным работам».

Руководитель практики _____ Е.Ю. Соболевская

Консультант практики _____ М.В. Водяницкий

Задание получил _____ Н.М. Малышев

КАЛЕНДАРНЫЙ ПЛАН-ГРАФИК (ДНЕВНИК)

прохождения учебной ознакомительной практики студента ВВГУ

Студент Малышев Никита Максимович направляется для прохождения учебной ознакомительной практики с 01.06.2026 г. по 03.07.2026 г.

№ п/п	Содержание выполняемых работ	Начало	Окончание	Заключение руководителя
1	Анализ задания, изучение теории графов и алгоритмов (BFS, DFS, Дейкстра, Прим, Краскал)	01.06.2026	05.06.2026	зачтено
2	Обзор существующих решений и сравнение технологий, обоснование выбора стека	08.06.2026	12.06.2026	зачтено
3	Проектирование архитектуры, реализация модели графа и структур данных	15.06.2026	19.06.2026	зачтено
4	Реализация алгоритмов, модуля визуализации и интерфейса	22.06.2026	26.06.2026	зачтено
5	Тестирование, отладка, оформление отчёта и подготовка к защите	29.06.2026	03.07.2026	зачтено

Согласовано:

Студент-практикант _____ Н.М. Малышев

Руководитель от кафедры _____ Е.Ю. Соболевская

Консультант от кафедры _____ М.В. Водяницкий

Содержание

Введение	5
1 Анализ предметной области и математический аппарат.....	7
1.1 Графы: основные понятия и определения	7
1.2 Алгоритмы обхода графа	7
1.3 Алгоритм Дейкстры.....	9
1.4 Минимальное остовное дерево.....	11
2 Обоснование выбора среды реализации	13
2.1 Сравнение технологий реализации	13
2.2 Обоснование выбора HTML, CSS и JavaScript	13
3 Проектирование и реализация приложения.....	15
3.1 Архитектура и модульная структура	15
3.2 Модель графа и структуры данных.....	16
3.3 Реализация алгоритмов	17
3.4 Интерактивный редактор графа	21
3.5 Система пошаговой визуализации	21
4 Тестирование и отладка	23
4.1 Инструмент тестирования и структура тестов.....	23
4.2 Примеры тестовых случаев	23
4.3 Граничные случаи.....	24
4.4 Результаты запуска тестов	24
4.5 Выявленные и устранённые дефекты	25
5 Руководство пользователя	27
Заключение.....	28
Список использованных источников.....	29
Приложение А Фрагменты исходного кода.....	30
Приложение Б Результаты запуска тестов	32

Введение

Учебная ознакомительная практика является неотъемлемой частью образовательного процесса по направлению подготовки 09.03.04 «Программная инженерия» во Владивостокском государственном университете. Основная цель практики – приобретение первичного профессионального опыта, закрепление и углубление теоретических знаний, полученных в процессе обучения, а также развитие практических навыков программирования и алгоритмического мышления.

В качестве индивидуального задания был выбран вариант 3 – «Визуализатор алгоритмов на графах». Тема актуальна, поскольку графовые алгоритмы широко применяются в маршрутизации компьютерных сетей, оптимизации транспортных потоков, анализе социальных связей и построении рекомендательных систем. Визуализация выполнения алгоритма позволяет наглядно показать его работу, что полезно как в учебных целях, так и при отладке.

Цель работы – разработать интерактивное веб-приложение для визуализации графовых алгоритмов, включающее реализацию обхода в ширину (BFS), обхода в глубину (DFS), алгоритма Дейкстры для поиска кратчайших путей, а также алгоритмов Прима и Краскала для построения минимального остовного дерева. Приложение должно предоставлять пользователю возможность интерактивного редактирования графа и пошагового наблюдения за выполнением выбранного алгоритма с подсветкой рёбер и вершин, выводом порядка обхода и суммарного веса построенного дерева.

Для достижения поставленной цели сформулированы следующие задачи:

- 1) изучить теоретические основы графовых алгоритмов и способов их визуализации;
- 2) проанализировать существующие решения и сравнить технологии реализации, обосновать выбор стека разработки;
- 3) спроектировать модульную архитектуру приложения и реализовать модель графа, структуры данных и алгоритмы;
- 4) разработать интерактивный редактор графа и систему пошаговой визуализации;
- 5) провести автоматизированное тестирование алгоритмов и структур данных, включая граничные случаи;
- 6) оформить отчёт по практике в соответствии с СК-СТО-ТР-04-1.005-2015 и опубликовать исходный код в репозитории Git.

Структура отчёта соответствует поставленным задачам. В первом разделе рассмотрен математический аппарат – основные понятия теории графов и теоретическое описание каждого алгоритма с блок-схемами. Во втором разделе проведено сравнение технологий реализации и обоснован выбор стека. Третий раздел посвящён проектированию и реализации

приложения с описанием модульной архитектуры. В четвёртом разделе представлены методика и результаты автоматизированного тестирования. Пятый раздел содержит руководство пользователя.

1 Анализ предметной области и математический аппарат

Графы – одно из фундаментальных понятий дискретной математики и информатики. Они моделируют парные отношения между объектами и применяются для представления транспортных и компьютерных сетей, социальных связей, молекулярных структур и многих других систем. Понимание графовых алгоритмов и умение их реализовывать – необходимая компетенция программного инженера. В рамках задания изучены пять классических алгоритмов; ниже приведены их теоретические основы и блок-схемы.

1.1 Графы: основные понятия и определения

Графом $G = (V, E)$ называется пара множеств, где V – непустое множество вершин, E – множество рёбер, каждое из которых соединяет пару вершин. Если рёбра имеют направление, граф называется ориентированным, иначе – неориентированным. Взвешенным называется граф, каждому ребру которого поставлено в соответствие числовое значение (вес), интерпретируемое как расстояние, стоимость или пропускная способность.

Существуют два основных способа представления графа в памяти. Матрица смежности – квадратная матрица размера $n \times n$, где элемент $a[i][j]$ равен весу ребра между вершинами i и j ; позволяет проверить наличие ребра за время $O(1)$, но требует $O(V^2)$ памяти и потому эффективна лишь для плотных графов. Список смежности – массив списков, где для каждой вершины хранятся её соседи; занимает $O(V + E)$ памяти и предпочтителен для разреженных графов. В разрабатываемом приложении используется список смежности, реализованный через хеш-таблицу (Map), что обеспечивает доступ к соседям вершины за время, пропорциональное её степени.

Путь в графе – последовательность вершин, в которой каждая пара соседних вершин соединена ребром. Цикл – путь, начинающийся и заканчивающийся в одной вершине. Граф называется связным, если между любыми двумя вершинами существует путь. Эти понятия лежат в основе рассматриваемых далее алгоритмов.

1.2 Алгоритмы обхода графа

Обход графа – систематическое посещение всех вершин путём перемещения по рёбрам. Существуют две основные стратегии обхода: в ширину (BFS) и в глубину (DFS). Оба алгоритма лежат в основе многих более сложных графовых алгоритмов.

1.2.1 Обход в ширину (BFS)

Алгоритм обхода в ширину использует очередь в качестве основной структуры данных. Стартовая вершина помещается в очередь и помечается как посещённая. На каждом шаге из очереди извлекается вершина, а все её непосещённые соседи помечаются и помещаются в очередь. Таким образом, BFS обходит граф по уровням: сначала посещаются

вершины на расстоянии 1 от стартовой, затем на расстоянии 2 и так далее. Временная сложность составляет $O(V + E)$, пространственная – $O(V + E)$. Блок-схема алгоритма приведена в соответствии с рисунком 1.

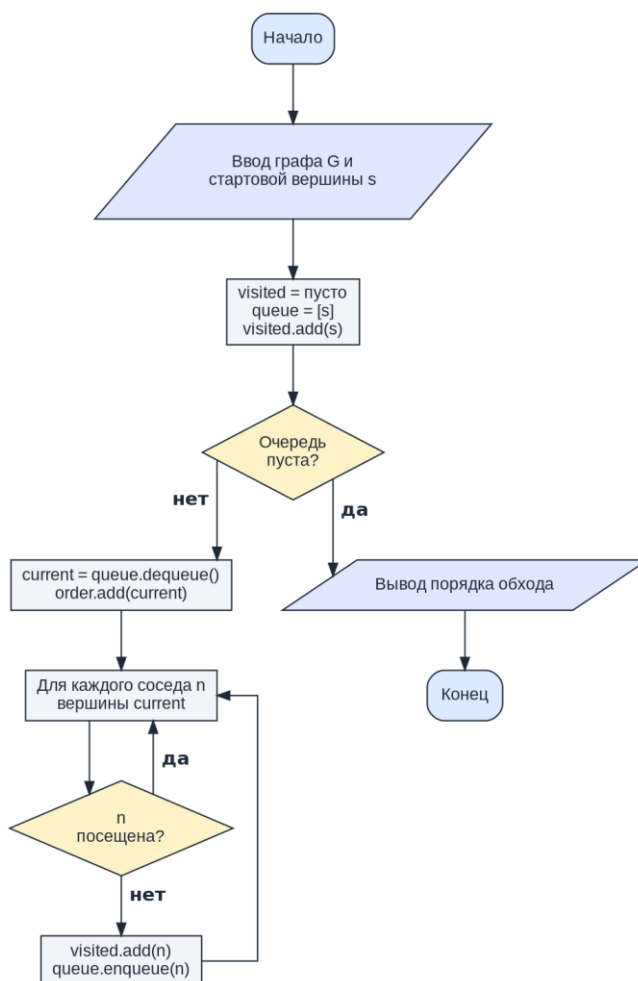


Рисунок 1 – Блок-схема алгоритма обхода в ширину (BFS)

1.2.2 Обход в глубину (DFS)

Алгоритм обхода в глубину использует стек (или рекурсию). Начиная со стартовой вершины, алгоритм переходит к одному из непосещённых соседей и углубляется до тех пор, пока не достигнет вершины без непосещённых соседей (тупика), после чего происходит возврат (backtracking) к предыдущей вершине. DFS приоритетно исследует одну ветвь до конца, прежде чем перейти к следующей. Временная сложность также составляет $O(V + E)$. DFS применяется в топологической сортировке, поиске циклов и компонент связности. Блок-схема алгоритма приведена в соответствии с рисунком 2.

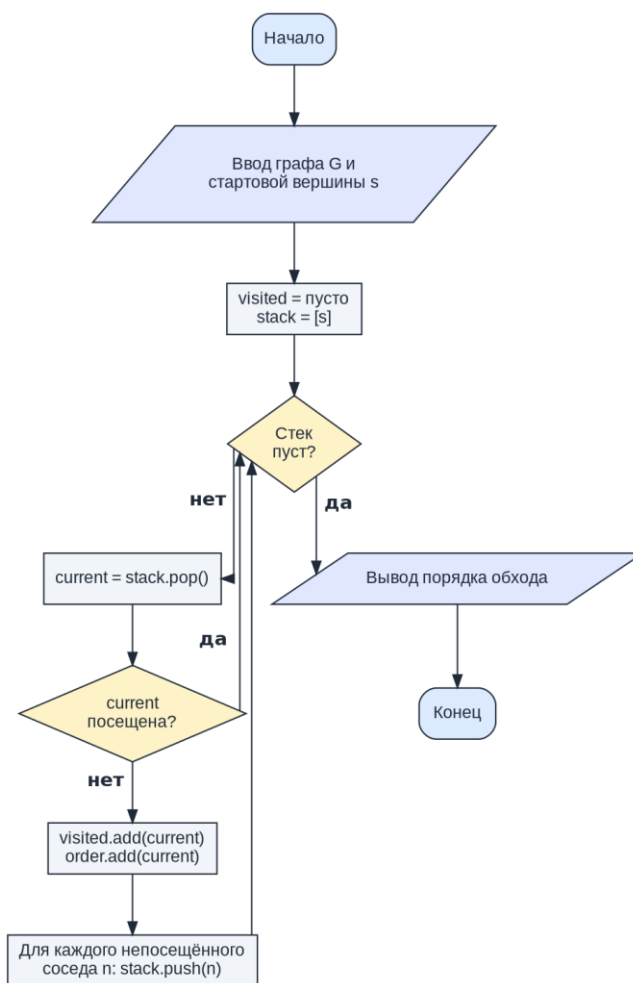


Рисунок 2 – Блок-схема алгоритма обхода в глубину (DFS)

Ключевое различие между BFS и DFS – в порядке посещения вершин: BFS расширяет фронт поиска равномерно, а DFS последовательно углубляется по одной ветви. Для визуализации это различие наглядно: при BFS видна «волна» обхода, при DFS – последовательное погружение с возвратами.

1.3 Алгоритм Дейкстры

Алгоритм Дейкстры предназначен для нахождения кратчайших путей от заданной стартовой вершины до всех остальных вершин во взвешенном графе с неотрицательными весами рёбер. Алгоритм разработан Э. Дейкстрой в 1956 году и широко применяется в маршрутизации и навигационных системах [8].

Каждой вершине ставится в соответствие метка – текущее наименьшее известное расстояние от стартовой вершины. Изначально всем вершинам назначается расстояние ∞ , а стартовой – 0. Для эффективного выбора вершины с минимальной меткой используется приоритетная очередь на основе бинарной кучи. На каждом шаге из очереди извлекается вершина u с минимальным расстоянием; если она уже посещалась, запись пропускается (ленивое удаление). Затем выполняется релаксация рёбер: для каждого непосещённого

соседа n проверяется, уменьшится ли расстояние до него через u , и если да – расстояние и предок обновляются, а новая пара (расстояние, n) помещается в очередь.

Применение бинарной кучи обеспечивает временную сложность $O((V + E) \log V)$: каждое ребро обрабатывается один раз ($O(E)$ релаксаций), а каждая операция с очередью – $O(\log V)$. Пространственная сложность составляет $O(V + E)$. Соседи вершины получаются из карты смежности за время, пропорциональное её степени, а не путём линейного просмотра всех рёбер. Блок-схема алгоритма приведена в соответствии с рисунком 3.

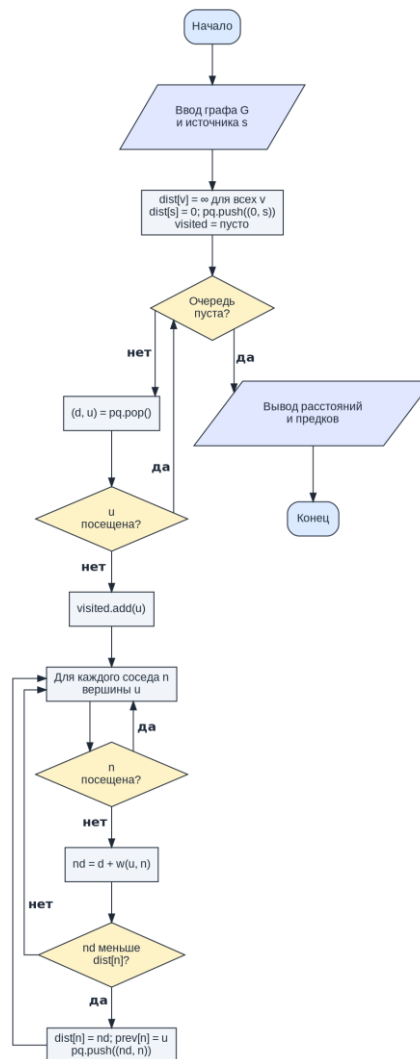


Рисунок 3 – Блок-схема алгоритма Дейкстры

Алгоритм Дейкстры корректен только для графов с неотрицательными весами. При наличии отрицательных весов следует использовать алгоритм Беллмана–Форда со сложностью $O(V \cdot E)$.

1.4 Минимальное остовное дерево

Минимальное остовное дерево (MST) связного взвешенного неориентированного графа – подмножество рёбер, соединяющее все вершины без циклов и имеющее

минимальную суммарную стоимость. MST применяется при проектировании сетей, кластеризации данных и в других задачах.

1.4.1 Алгоритм Прима

Алгоритм Прима строит MST, начиная с произвольной вершины. На каждом шаге к дереву добавляется ребро минимального веса, соединяющее вершину дерева с вершиной, ещё не включённой в него. Для выбора ребра используется приоритетная очередь, что даёт сложность $O((V + E) \log V)$. Блок-схема приведена в соответствии с рисунком 4.

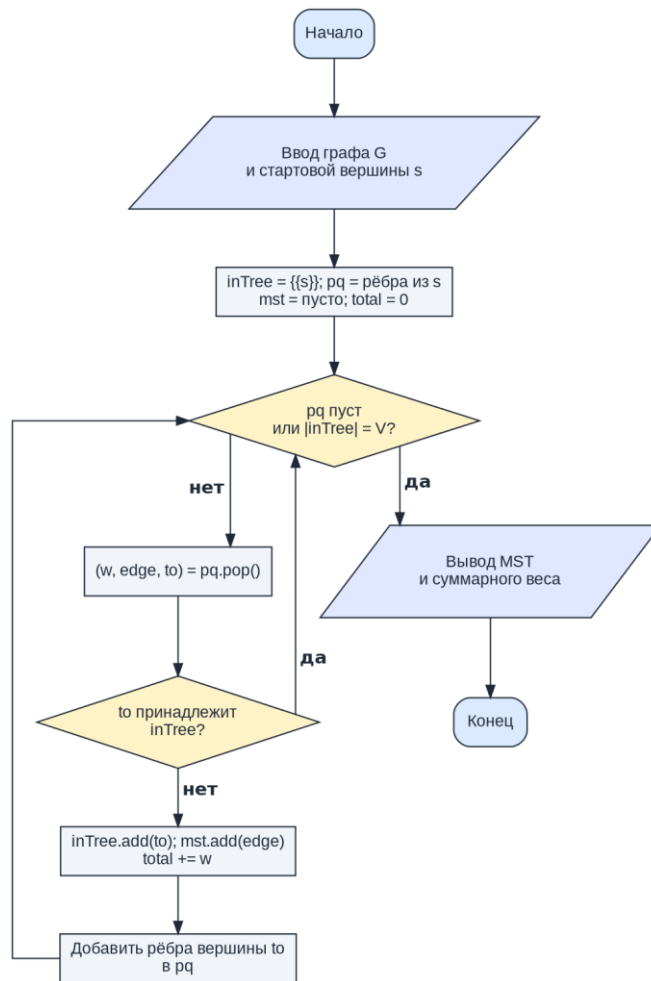


Рисунок 4 – Блок-схема алгоритма Прима

1.4.2 Алгоритм Краскала

Алгоритм Краскала сортирует все рёбра по возрастанию веса и последовательно добавляет ребро в остов, если оно не образует цикла с уже выбранными рёбрами. Проверка цикла выполняется через структуру данных «система непересекающихся множеств» (DSU, Disjoint Set Union): если обе вершины ребра уже принадлежат одной компоненте ($\text{find}(a) = \text{find}(b)$), ребро образует цикл и отбрасывается. DSU реализовано с эвристиками сжатия пути и объединения по рангу, что делает операции практически константными. Общая сложность – $O(E \log E)$ и определяется сортировкой рёбер. Блок-схема приведена в соответствии с рисунком 5.

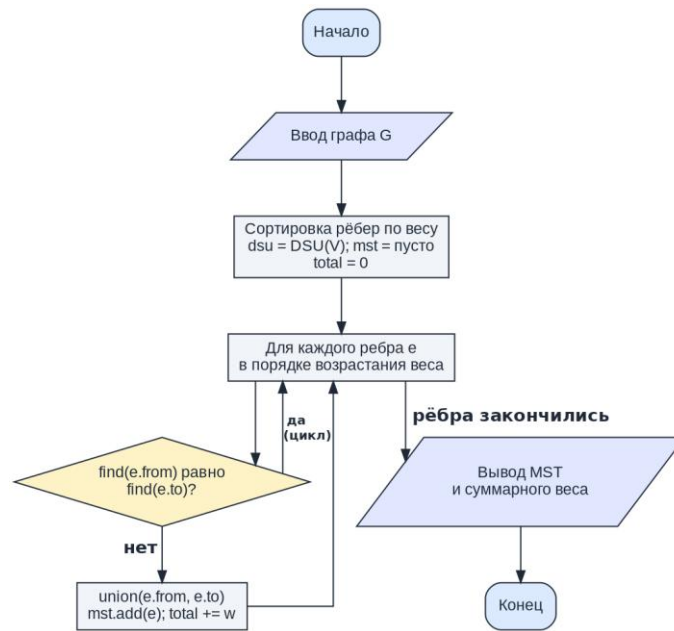


Рисунок 5 – Блок-схема алгоритма Краскала

Важно отметить свойство корректности DSU: проверка цикла выполняется по принадлежности вершин компонентам и не зависит от веса ребра или порядка рёбер с равными весами. Порядок рёбер с равными весами может повлиять лишь на выбор конкретного MST, если таких деревьев несколько, но не на сам факт обнаружения цикла. Это свойство проверено отдельными тестами (см. подраздел 4.3).

2 Обоснование выбора среды реализации

Выбор технологии реализации – важный этап, от которого зависят производительность, удобство демонстрации и кроссплатформенность приложения. Были рассмотрены несколько подходов; результаты сравнения по установленным критериям приведены в таблице 1.

2.1 Сравнение технологий реализации

Для сравнения выбраны шесть критериев: кроссплатформенность, требования к установке и зависимостям, возможности визуализации и анимации, условия лицензирования, кривая обучения и доступность (возможность запуска без установки на любом устройстве). Рассмотрены пять вариантов реализации.

Таблица 1 – Сравнение технологий реализации визуализатора

Критерий	Python + Tkinter	Python + PyQt	C++ + Qt	HTML/CSS/JS
Кроссплатформенность	Частичная	Высокая	Высокая	Полная (браузер)
Установка и зависимости	Нужен Python	Нужен Python + PyQt	Нужен компилятор, Qt	Не требуется
Визуализация и анимация	Ограниченные	Хорошие	Хорошие	SVG + CSS-переходы
Лицензирование	Python License (свободная)	GPL / LGPL (PySide6)	LGPL / коммерческая	Открытые стандарты W3C
Кривая обучения	Низкая	Средняя	Высокая	Низкая–средняя
Запуск без установки	Нет	Нет	Нет	Да (браузер)
Итог	Не подходит	Подходит	Избыточен	Выбрано

Анализ показывает, что веб-технологии (HTML, CSS, JavaScript) обладают совокупностью преимуществ: не требуют установки, обеспечивают полную кроссплатформенность и позволяют использовать SVG для масштабируемой векторной графики с плавными CSS-анимациями. Desktopные решения на Python (Tkinter, PyQt) и C++ требуют установки среды выполнения и зависимостей, что усложняет демонстрацию результатов. Отметим также, что PyQt распространяется по лицензии GPL, а PySide6 – по LGPL; обе библиотеки доступны для свободного использования в учебных проектах.

2.2 Обоснование выбора HTML, CSS и JavaScript

На основании сравнения принято решение реализовать визуализатор в виде веб-приложения на чистых HTML, CSS и JavaScript без использования сторонних фреймворков. Выбор обоснован следующими преимуществами.

Во-первых, приложение не требует установки: для запуска достаточно открыть его в любом современном браузере. Это упрощает демонстрацию на защите и распространение. Во-вторых, технология SVG представляет каждый элемент графа как отдельный DOM-узел, к которому применимы CSS-переходы, что обеспечивает плавную анимацию подсветки без сторонних библиотек. В-третьих, JavaScript обеспечивает достаточную производительность для графов учебного масштаба, а встроенные средства разработчика браузера облегчают отладку. В-четвёртых, стандартные веб-технологии гарантируют полную кроссплатформенность.

Использование полнофункционального фреймворка (React, Vue.js) признано избыточным: задача не требует сложной маршрутизации или серверной части. Вместе с тем, чтобы избежать монолитной структуры, проект разделён на ES-модули с чётким разделением ответственности (см. раздел 3).

3 Проектирование и реализация приложения

В данном разделе описана архитектура приложения, реализация модели графа и структур данных, алгоритмов, редактора графа и системы визуализации. Исходный код опубликован в репозитории Git и снабжён инструкцией по запуску (README.md); приложение также развёрнуто через GitHub Pages по адресу <https://tartaletka03.github.io/graph-visualizer/>.

3.1 Архитектура и модульная структура

Приложение построено по модульному принципу с использованием ES-модулей. Каждый модуль имеет единственную ответственность и взаимодействует с другими через чётко определённые интерфейсы. Структура модулей приведена в соответствии с рисунком 6.

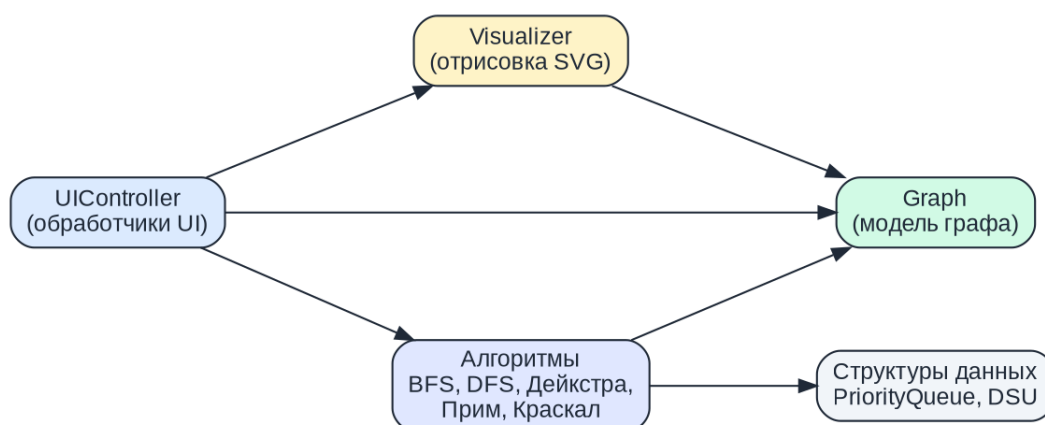


Рисунок 6 – Архитектура приложения (модульная структура)

Модули разделены на пять групп:

- модель графа (core/Graph.js) – единый источник данных о структуре графа; хранит вершины и рёбра и поддерживает карту смежности;
- структуры данных (structures/) – PriorityQueue (бинарная min-куча) и DSU (система непересекающихся множеств);
- алгоритмы (algorithms/) – пять чистых функций, каждая возвращает массив шагов – описаний состояний графа;
- визуализация (visualization/Visualizer.js) – отрисовка графа на SVG и применение шагов;
- контроллер интерфейса (ui/UIController.js) – связывает модель, визуализатор и алгоритмы с действиями пользователя.

Направления связей между модулями, показанные на рисунке 6: контроллер интерфейса запускает алгоритмы, применяет их шаги через визуализатор и редактирует

модель графа; алгоритмы читают структуру графа из модели и используют структуры данных; визуализатор отрисовывает граф, читая данные из модели. Таким образом, модель графа является центральным источником данных, а контроллер интерфейса – координатором взаимодействия.

Такое разделение позволяет тестировать алгоритмы и структуры данных изолированно, без браузерного окружения, а также изменять визуализацию, не затрагивая логику алгоритмов. Интерфейс приложения разделён на боковую панель управления и рабочую область с SVG-холстом; общий вид приведён в соответствии с рисунком 7.

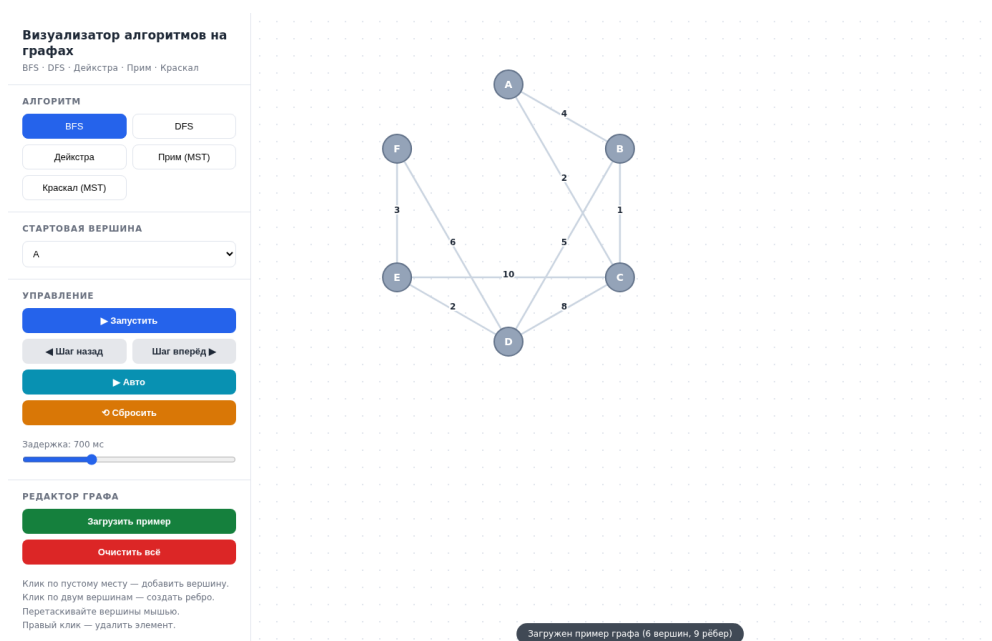


Рисунок 7 – Интерфейс приложения с загруженным примером графа

На рисунке 7 видно, что боковая панель слева содержит кнопки выбора алгоритма, управления выполнением и редактирования графа, а справа расположен SVG-холст с графом из шести вершин. Информационная панель внизу слева отображает описание текущего шага и результаты выполнения алгоритма.

3.2 Модель графа и структуры данных

Класс `Graph` (`core/Graph.js`) хранит вершины и рёбра в хеш-таблицах `Map` и поддерживает карту смежности – отображение идентификатора вершины в список её соседей. Карта обновляется при добавлении и удалении рёбер, благодаря чему метод `getNeighbors` возвращает соседей за время, пропорциональное степени вершины, а не просматривает весь массив рёбер. Это устраняет узкое место, характерное для наивной реализации, и снижает фактическую сложность алгоритмов.

Приоритетная очередь `PriorityQueue` (`structures/PriorityQueue.js`) реализована как бинарная `min`-куча на массиве. Операции `push` и `pop` выполняются за $O(\log n)$ с

восстановлением свойства кучи просеиванием вверх и вниз. В алгоритме Дейкстры применяется ленивое удаление: при обновлении расстояния до вершины новая пара (расстояние, вершина) просто добавляется в кучу, а устаревшие записи с большим расстоянием пропускаются при извлечении. Это избавляет от реализации сложной операции decrease-key.

Система непересекающихся множеств DSU (structures/DSU.js) реализована с эвристиками сжатия пути в find и объединения по рангу в union. Операция union связывает корни множеств ($\text{parent}[\text{find}(a)] = \text{find}(b)$), а не сами вершины – это ключевое требование корректности: связывание вершин напрямую приводит к ошибкам проверки циклов.

3.3 Реализация алгоритмов

Каждый алгоритм реализован как чистая функция, принимающая граф (и стартовую вершину при необходимости) и возвращающая массив шагов. Шаг – это снимок состояния: текущая вершина, множества посещённых вершин и активных рёбер, расстояния (для Дейкстры), рёбра MST и суммарный вес (для Прима и Краскала). Единый формат шагов позволяет визуализатору единообразно отображать любой алгоритм.

3.3.1 Обход в ширину (BFS)

Алгоритм использует очередь. Стартовая вершина помещается в очередь и помечается посещённой. На каждом шаге из очереди извлекается вершина, а её непосещённые соседи помечаются и помещаются в очередь. Соседи получаются через карту смежности. Сложность – $O(V + E)$. Пошаговая визуализация приведена в соответствии с рисунком 8.

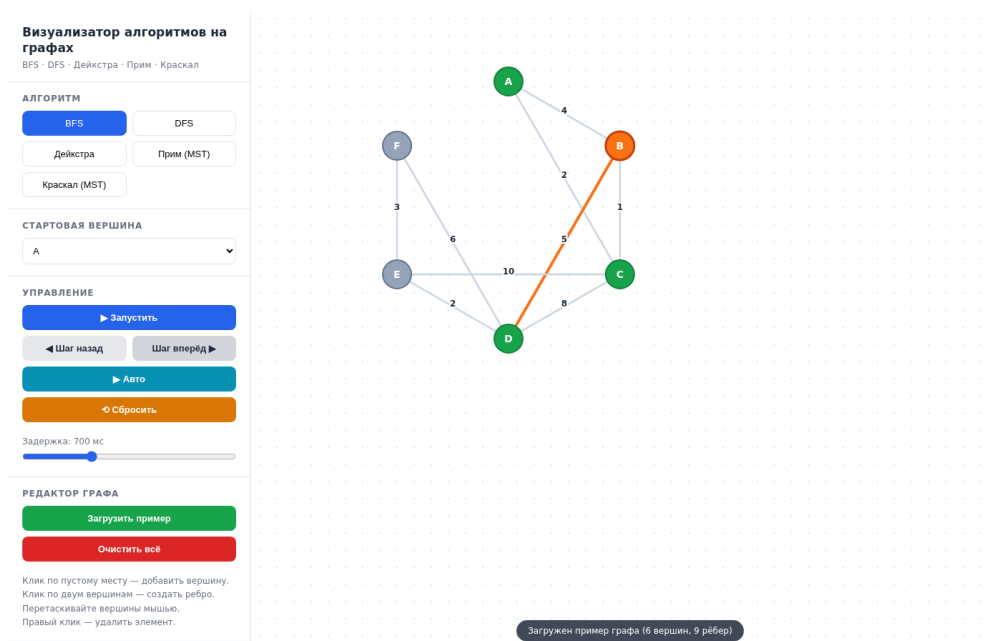


Рисунок 8 – Пошаговая визуализация алгоритма BFS

На рисунке 8 показан момент выполнения обхода в ширину: текущая вершина подсвечена оранжевым, посещённые – зелёным, вершины в очереди – голубым. Информационной панели отображается порядок обхода и описание текущего шага.

3.3.2 Обход в глубину (DFS)

Алгоритм реализован итеративно на явном стеке (без рекурсии), что позволяет генерировать шаги единообразно и избегать переполнения стека вызовов. Соседи помещаются в стек в обратном порядке для детерминированного обхода. Сложность – $O(V + E)$. Пошаговая визуализация приведена в соответствии с рисунком 9.

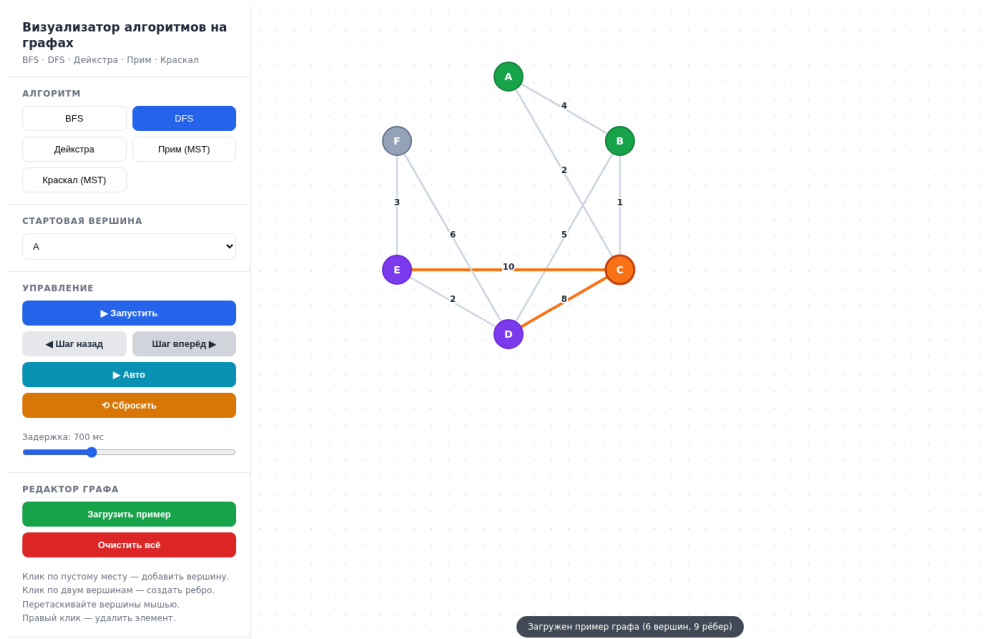


Рисунок 9 – Пошаговая визуализация алгоритма DFS

На рисунке 9 виден момент углубления в одну из ветвей графа: алгоритм последовательно посещает вершины вдоль выбранного пути, а вершины в стеке подсвечены фиолетовым. Возврат из тупика отображается как переход к следующей вершине из стека.

3.3.3 Алгоритм Дейкстры

Алгоритм использует приоритетную очередь на бинарной куче. Изначально расстояния до всех вершин равны ∞ , расстояние до стартовой – 0; в очередь помещается пара $(0, \text{start})$. На каждой итерации из очереди извлекается вершина с минимальным расстоянием; если она уже посещалась, запись пропускается (ленивое удаление). Для каждого непосещённого соседа выполняется релаксация: если путь через текущую вершину короче, расстояние и предок обновляются, а новая пара помещается в очередь. Сложность – $O((V + E) \log V)$. Пошаговая визуализация с таблицей расстояний приведена в соответствии с рисунком 10.

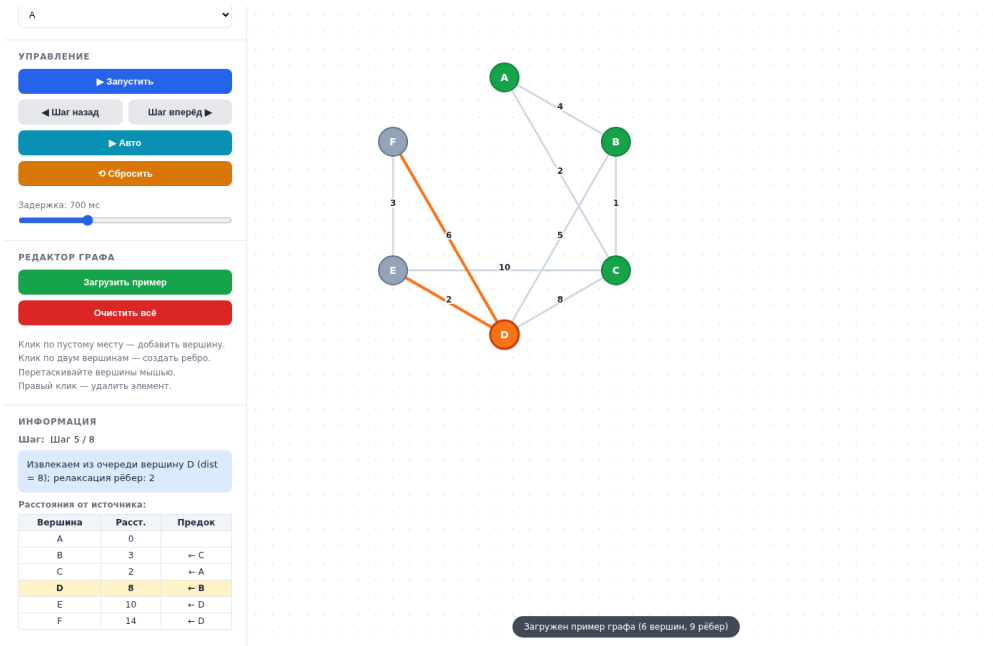


Рисунок 10 – Пошаговая визуализация алгоритма Дейкстры с таблицей расстояний

В информационной панели отображается таблица расстояний от источника, обновляющаяся на каждой итерации, а также предок каждой вершины для восстановления кратчайшего пути. Пример обновления расстояний по итерациям для графа из шести вершин (A–F) при старте из A приведён в таблице 2.

Таблица 2 – Обновление расстояний по итерациям алгоритма Дейкстры (старт из A)

Итерация	Извлечена	A	B	C	D	E	F
0 (init)	–	0	∞	∞	∞	∞	∞
1	A (0)	0	4	2	∞	∞	∞
2	C (2)	0	3	2	10	12	∞
3	B (3)	0	3	2	8	12	∞
4	D (8)	0	3	2	8	10	14
5	E (10)	0	3	2	8	10	13
6	F (13)	0	3	2	8	10	13

В таблице 2 знаком ∞ обозначено бесконечное расстояние. На итерации 2 при обработке вершины C расстояние до B уменьшилось с 4 до 3 (путь A–C–B короче прямого A–B). Итоговые кратчайшие расстояния: A = 0, B = 3, C = 2, D = 8, E = 10, F = 13. Кратчайший путь до F: A → C → B → D → E → F с суммарным весом 2 + 1 + 5 + 2 + 3 = 13. Этот результат подтверждается автоматизированным тестом (см. раздел 4).

3.3.4 Алгоритм Прима

Алгоритм начинает со стартовой вершины и на каждом шаге добавляет ребро минимального веса, соединяющее дерево с новой вершиной. Кандидаты хранятся в

приоритетной очереди; ребро, ведущее в уже включённую вершину, пропускается. Сложность – $O((V + E) \log V)$. Пошаговая визуализация приведена в соответствии с рисунком 11.

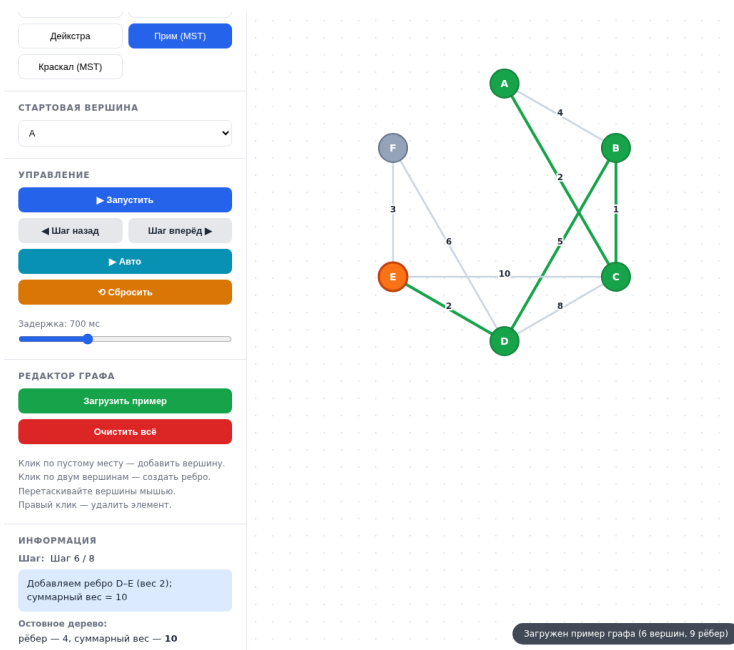


Рисунок 11 – Пошаговая визуализация алгоритма Прима

На рисунке 11 показан момент построения остова: рёбра, уже включённые в MST, подсвечены зелёным, а текущее рассматриваемое ребро – оранжевым. Суммарный вес дерева отображается в информационной панели и увеличивается по мере добавления рёбер.

3.3.5 Алгоритм Краскала

Алгоритм сортирует все рёбра графа по возрастанию веса и последовательно рассматривает их в полученном порядке. Для каждого ребра проверяется, находятся ли его концы в одной компоненте связности (для этого используется структура DSU): если да, то добавление ребра привело бы к образованию цикла, и оно отбрасывается; если нет, компоненты объединяются, а ребро включается в остова. Процесс продолжается до тех пор, пока не будут рассмотрены все рёбра или пока в остова не будет включено $V-1$ рёбер (что достаточно для связного графа).

Эффективность алгоритма Краскала во многом определяется скоростью операций DSU. С эвристиками сжатия пути и объединения по рангу амортизированная сложность каждой операции близка к $O(1)$, поэтому общая сложность алгоритма составляет $O(E \log E)$ и определяется временем сортировки рёбер. Пошаговая визуализация алгоритма Краскала приведена в соответствии с рисунком 12; отброшенные рёбра, образующие цикл, отображаются пунктиром.

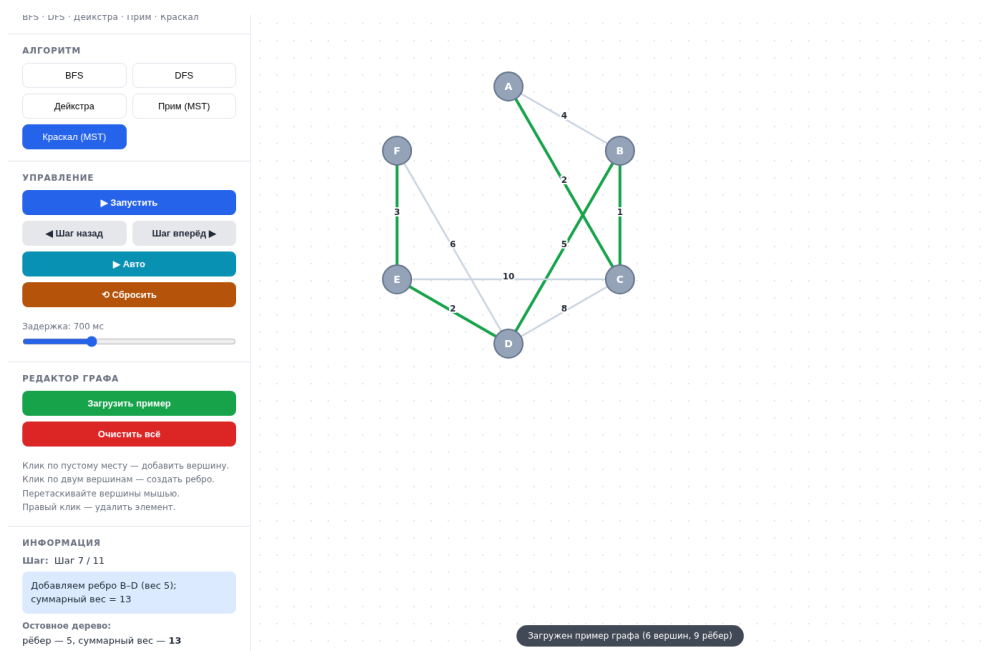


Рисунок 12 – Пошаговая визуализация алгоритма Краскала

Для тестового графа из шести вершин оба алгоритма построения MST (Прим и Краскал) дают одинаковый суммарный вес – 13. Состав рёбер MST: B–C (1), A–C (2), D–E (2), E–F (3), B–D (5). Результат не зависит от выбора стартовой вершины для алгоритма Прима, а также от порядка рёбер с равными весами в алгоритме Краскала (изменение порядка влияет лишь на выбор конкретного MST, когда их несколько, но не на суммарный вес).

3.4 Интерактивный редактор графа

Редактор графа позволяет создавать и модифицировать граф в браузере. Поддерживаемые операции: добавление вершины щелчком по пустой области холста; создание взвешенного ребра последовательными щелчками по двум вершинам с вводом веса через модальное окно; перемещение вершин перетаскиванием; удаление вершин и рёбер через контекстное меню. Предусмотрена загрузка примера графа из шести вершин и девяти рёбер, а также полная очистка холста.

3.5 Система пошаговой визуализации

Центральная часть визуализатора – пошаговое выполнение алгоритмов. Каждый алгоритм возвращает массив шагов – snapshots состояния графа, где каждый шаг содержит описание текущего действия, множество активных (подсвечиваемых) вершин и рёбер, а также дополнительную информацию: порядок обхода, обновлённые расстояния или текущий вес остовного дерева. Такой подход позволяет отделить логику алгоритма от визуального представления: алгоритм только вычисляет последовательность состояний, а визуализатор последовательно применяет их к SVG-элементам.

При нажатии кнопки «Шаг вперёд» визуализатор считывает следующий шаг из массива и применяет изменения к холсту, назначая элементам классы состояний. Это обеспечивает плавную анимацию: цвет вершины или ребра изменяется постепенно благодаря CSS-переходам. Информационная панель синхронно обновляется, отображая описание текущего шага и промежуточные результаты. Цвета состояний, используемые для подсветки вершин и рёбер, приведены в таблице 3.

Таблица 3 – Состояния вершин и рёбер при визуализации

Объект	Состояние	Цвет	Значение
Вершина	Текущая	Оранжевый	Обрабатывается на данном шаге
Вершина	Посещённая	Зелёный	Включена в результат обхода
Вершина	В очереди/стеке	Голубой/фиолетовый	Запланирована к обработке
Вершина	В дереве (MST)	Зелёный	Включена в остовное дерево
Ребро	Активное	Оранжевый	Рассматривается на шаге
Ребро	В остове	Зелёный	Включено в MST
Ребро	Отброшенное	Серый пунктир	Образует цикл (Краскал)

Плавность переходов обеспечивается CSS-свойством transition длительностью 300 мс. Предусмотрено автоматическое воспроизведение с регулируемой задержкой (от 100 до 2000 мс) и сброс визуализации без удаления графа.

4 Тестирование и отладка

Тестирование направлено на проверку корректности модели графа, структур данных и всех алгоритмов, включая граничные случаи. Тесты автоматизированы и публикуются в репозитории вместе с исходным кодом.

4.1 Инструмент тестирования и структура тестов

В качестве инструмента используется лёгкий собственный тест-фреймворк (tests/runner.mjs), реализующий функции describe, it и expect с набором проверок (toEqual, toBe, toContain, toThrow и др.). Фреймворк не имеет внешних зависимостей и запускается как в Node.js (команда node tests/run-all.mjs), так и в браузере (файл tests/index.html). Алгоритмы и структуры данных реализованы как чистые ES-модули без обращения к браузерным API, что позволяет тестировать их изолированно. Структура тестов повторяет структуру модулей; общие тестовые графы и эталонные результаты вынесены в файл fixtures.mjs.

4.2 Примеры тестовых случаев

В таблице 4 приведены конкретные тестовые случаи с входными данными, ожидаемым и фактическим результатом. В качестве входного графа используется канонический граф из шести вершин. Для каждого алгоритма проверяется как корректность результата (порядок обхода, расстояния, состав MST), так и согласованность с эталонными значениями, вычисленными вручную.

Таблица 4 – Примеры тестовых случаев

№	Алгоритм	Входные данные	Ожидаемый результат	Фактический результат
1	BFS	Граф 6 вершин, старт A	Порядок: A, B, C, D, E, F	A, B, C, D, E, F
2	DFS	Граф 6 вершин, старт A	Посещены все 6 вершин	6 вершин
3	Дейкстра	Граф 6 вершин, старт A	A=0, B=3, C=2, D=8, E=10, F=13	0, 3, 2, 8, 10, 13
4	Дейкстра	Путь к F	A→C→B→D→E→F (вес 13)	A→C→B→D→E→F
5	Прим	Граф 6 вершин, старт A	Вес MST = 13, 5 рёбер	13, 5 рёбер
6	Краскал	Граф 6 вершин	Вес MST = 13, 5 рёбер	13, 5 рёбер
7	Краскал	Состав MST	BC=1, AC=2, DE=2, EF=3, BD=5	Совпало
8	DSU	union(0,1), union(1,2), ребро 0-2	Цикл обнаружен	Цикл обнаружен

Все перечисленные тестовые случаи проходят успешно. Полный набор включает 57 проверок, охватывающих модель графа, приоритетную очередь, DSU и все пять алгоритмов.

4.3 Граничные случаи

Отдельно проверены граничные случаи, в которых чаще всего возникают ошибки: граф из одной вершины (алгоритмы не падают); несвязный граф (BFS и DFS посещают только достижимую компоненту, Краскал строит остовный лес); изолированная вершина (расстояние до неё в Дейкстре остаётся ∞); равные веса рёбер (треугольник и квадрат с диагональю дают корректное MST, циклы обнаруживаются независимо от порядка рёбер); граф без рёбер (MST пуст); попытка создать петлю или ребро между несуществующими вершинами (модель выбрасывает исключение).

Граничные тесты подтверждают, что обнаружение цикла в Краскале зависит только от принадлежности вершин компонентам и не зависит от порядка рёбер с равными весами – это соответствует свойству корректного DSU.

4.4 Результаты запуска тестов

Сводные результаты запуска тестов приведены в таблице 5.

Таблица 5 – Результаты запуска автоматизированных тестов

Модуль	Число тестов	Пройдено	Не пройдено
Модель графа	11	11	0
Приоритетная очередь	5	5	0
Система непересекающихся множеств	6	6	0
BFS	6	6	0
DFS	5	5	0
Дейкстра	8	8	0
Прим	6	6	0
Краскал	10	10	0
Итого	57	57	0

Из таблицы 5 видно, что все 57 тестов завершаются успешно: каждый модуль показывает нулевое количество непройденных проверок. Это подтверждает корректность реализации модели графа, структур данных и всех пяти алгоритмов на рассмотренном наборе тестовых случаев, включая граничные сценарии. Визуальное представление результатов запуска тестов в браузерном тест-раннере приведено в соответствии с рисунком 13.



Рисунок 13 – Результаты запуска автоматизированных тестов

4.5 Выявленные и устранённые дефекты

В ходе разработки и тестирования выявлены и устранены следующие дефекты.

Дефект в реализации DSU. В ранней версии алгоритма Краскала операция объединения множеств устанавливала связь между самими вершинами ($\text{parent}[a] = b$), а не между корнями их множеств. Из-за этого проверка $\text{find}(a) = \text{find}(b)$ возвращала некорректный результат для вершин, уже находившихся в одной компоненте: в ряде случаев ребро, замыкающее цикл, ошибочно добавлялось в остов. Дефект устранён приведением операции объединения к виду $\text{parent}[\text{find}(a)] = \text{find}(b)$ с добавлением эвристики сжатия пути и объединения по рангу.

Отдельно следует ответить на вопрос о влиянии порядка рёбер с равными весами. Изменение порядка рёбер с равными весами (например, сортировка по вторичному ключу) не может устранить цикл, поскольку DSU проверяет принадлежность вершин компонентам независимо от веса ребра. Порядок рёбер с равными весами влияет лишь на выбор конкретного MST, когда таких деревьев несколько. Таким образом, первоначальная интерпретация дефекта как «проблемы равных весов» была ошибочной; истинной причиной была некорректная реализация операции union.

Дефект доступа к соседям. В наивной реализации метод `getNeighbors` просматривал весь массив рёбер, что повышало фактическую сложность алгоритмов до $O(V^2 + V \cdot E)$. Устранён введением карты смежности, обновляемой при изменении набора рёбер.

Несоответствие заявленной сложности алгоритма Дейкстры. В ранней версии для выбора вершины с минимальным расстоянием использовался линейный поиск по всем вершинам (сложность $O(V^2)$), тогда как в описании заявлялась бинарная куча. Дефект устранён заменой линейного поиска на приоритетную очередь на бинарной куче с ленивым удалением, что обеспечивает заявленную сложность $O((V + E) \log V)$.

Дефект отрисовки. При определённых конфигурациях графа подписи весов рёбер перекрывались вершинами. Устранён изменением порядка отрисовки слоёв (рёбра, подписи, вершины) и добавлением белой обводки текста (`paint-order: stroke`).

5 Руководство пользователя

Приложение опубликовано в репозитории Git и развёрнуто для онлайн-доступа через GitHub Pages по адресу: <https://tartaletka03.github.io/graph-visualizer/>. Открыть приложение можно в любом современном браузере (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) без установки дополнительного программного обеспечения. Этот способ запуска является основным и наиболее удобным.

Локальный запуск возможен двумя способами. Первый – открыть файл `index.html` непосредственно в браузере. Поскольку приложение использует ES-модули, при открытии файла через протокол `file://` некоторые браузеры ограничивают загрузку модулей; в этом случае следует запустить локальный веб-сервер, например командой `python3 -m http.server 8000` из корня проекта, и открыть адрес `http://localhost:8000`. Второй способ – клонировать репозиторий Git и запустить любой статический веб-сервер. Подробная инструкция приведена в файле `README.md` репозитория.

Для быстрого старта нажмите кнопку «Загрузить пример» – будет создан граф из шести вершин и девяти рёбер. Для создания собственного графа щёлкните левой кнопкой мыши по пустой области холста (добавление вершины), затем по двум вершинам подряд (создание ребра с вводом веса в модальном окне). Вершины можно перетаскивать; для удаления вершины или ребра используйте контекстное меню по правому щелчку.

Для запуска визуализации выберите алгоритм в боковой панели, при необходимости укажите стартовую вершину и нажмите «Запустить». Далее используйте кнопку «Шаг вперёд» для пошагового выполнения или «Авто» для автоматического воспроизведения с задержкой, регулируемой слайдером. Кнопка «Сбросить» возвращает граф в исходное состояние без удаления структуры.

В информационной панели отображаются: описание текущего шага, порядок обхода (для BFS и DFS), таблица расстояний и предков (для Дейкстры), количество рёбер и суммарный вес остовного дерева (для Прима и Краскала).

Для запуска автоматизированных тестов выполните в терминале команду `node tests/run-all.mjs` (требуется Node.js версии 14 и выше) или откройте файл `tests/index.html` в браузере через локальный сервер.

Заключение

В ходе учебной ознакомительной практики выполнено индивидуальное задание по разработке визуализатора алгоритмов на графах. Поставленная цель достигнута, все сформулированные задачи решены.

Проведён анализ предметной области: изучены основные понятия теории графов и пять алгоритмов – BFS, DFS, Дейкстра, Прим и Краскал. Для каждого алгоритма составлены блок-схемы по ГОСТ 19.701–90 и обоснована вычислительная сложность.

На основании сравнения технологий по установленным критериям выбран стек HTML, CSS и JavaScript. Рассмотренные альтернативы (Tkinter, PyQt, C++ с Qt) доступны по открытым лицензиям, однако требуют установки среды выполнения и зависимостей, тогда как веб-приложение запускается в браузере без установки.

Разработано веб-приложение с модульной архитектурой на ES-модулях. Структура разделена на модель графа, структуры данных (бинарная куча и DSU), алгоритмы, визуализацию и контроллер интерфейса. Алгоритм Дейкстры использует приоритетную очередь на бинарной куче с ленивым удалением (сложность $O((V + E) \log V)$); доступ к соседям реализован через карту смежности. Реализация DSU корректна: операция `union` связывает корни множеств, что исключает пропуски циклов.

Разработаны автоматизированные тесты (57 проверок) на собственном тест-фреймворке, охватывающие штатные и граничные случаи, включая равные веса рёбер и несвязные графы. Все тесты проходят успешно. Выявленные дефекты – некорректная реализация `union` в DSU, линейный поиск минимума в Дейкстре и просмотр всего массива рёбер при получении соседей – устранены.

Исходный код опубликован в репозитории Git и снабжён инструкцией по запуску (README.md); приложение развёрнуто через GitHub Pages и доступно онлайн. В результате приобретён опыт самостоятельной профессиональной деятельности: от анализа задачи и выбора технологий до разработки, тестирования и документирования программного продукта. Полученные навыки станут основой для дальнейшего изучения специальных дисциплин и выполнения курсовых и выпускных квалификационных работ.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Кормен, Т. Алгоритмы: построение и анализ = Introduction to Algorithms / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. – 3-е изд. – Москва : Вильямс, 2020. – 1328 с.
- 2 Седжвик, Р. Алгоритмы на графах / Р. Седжвик. – Москва : Диалектика, 2018. – 400 с.
- 3 Батищев, Р. В. Структуры и алгоритмы обработки данных. Иерархические структуры и графы : методические указания / Р. В. Батищев. – Липецк : Изд-во ЛГТУ, 2017. – 35 с. – URL: <https://lib.rucont.ru/efd/651980> (дата обращения: 19.01.2025).
- 4 Кривцов, А. Н. Технологии программирования. Технология программирования на C/C++ : учебное пособие / А. Н. Кривцов. – Санкт-Петербург : СПбГУТ им. М. А. Бонч-Бруевича, 2021. – 274 с. – URL: <https://e.lanbook.com/book/279680> (дата обращения: 17.06.2025).
- 5 Шелудько, В. М. Основы программирования на языке высокого уровня Python : учебное пособие / В. М. Шелудько. – Ростов-на-Дону : Изд-во ЮФУ, 2017. – 148 с. – URL: <https://lib.rucont.ru/efd/692437> (дата обращения: 19.01.2025).
- 6 Колдаев, В. Д. Основы алгоритмизации и программирования : учебное пособие / В. Д. Колдаев ; под ред. Л. Г. Гагариной. – Москва : ФОРУМ : ИНФРА-М, 2022. – 414 с.
- 7 MDN Web Docs. SVG: Scalable Vector Graphics [Электронный ресурс] // Mozilla Developer Network. – URL: <https://developer.mozilla.org/ru/docs/Web/SVG> (дата обращения: 01.06.2026).
- 8 Dijkstra, E. W. A note on two problems in connexion with graphs / E. W. Dijkstra // Numerische Mathematik. – 1959. – Vol. 1. – P. 269–271.
- 9 Prim, R. C. Shortest connection networks and some generalizations / R. C. Prim // Bell System Technical Journal. – 1957. – Vol. 36, No. 6. – P. 1389–1401.
- 10 Kruskal, J. B. On the shortest spanning subtree of a graph and the traveling salesman problem / J. B. Kruskal // Proceedings of the American Mathematical Society. – 1956. – Vol. 7, No. 1. – P. 48–50.
- 11 Tarjan, R. E. Efficiency of a good but not linear set union algorithm / R. E. Tarjan // Journal of the ACM. – 1975. – Vol. 22, No. 2. – P. 215–225.
- 12 ГОСТ 19.701–90. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. – Москва : Стандартинформ, 2010. – 26 с.
- 13 ECMA-262. ECMAScript 2023 Language Specification [Электронный ресурс]. – 14th ed. – URL: <https://tc39.es/ecma262/> (дата обращения: 01.06.2026).
- 14 Git – Documentation [Электронный ресурс]. – URL: <https://git-scm.com/doc> (дата обращения: 01.06.2026).

Приложение А

Фрагменты исходного кода

А.1 Класс Graph – карта смежности (core/Graph.js)

```
class Graph {
  constructor(directed = false) {
    this.directed = directed;
    this.vertices = new Map();
    this.edges = new Map();
    this._adjacency = new Map();    // vertexId -> соседи
  }
  getNeighbors(vertexId) {
    const list = this._adjacency.get(vertexId);
    return list ? list.slice() : [];    // O(степень), не O(E)
  }
}
```

А.2 Бинарная min-куча – извлечение минимума (structures/PriorityQueue.js)

```
class PriorityQueue {
  pop() {
    if (this._heap.length === 0) return undefined;
    const top = this._heap[0];
    const last = this._heap.pop();
    if (this._heap.length > 0) {
      this._heap[0] = last;
      this._siftDown(0);    // O(log n)
    }
    return top;
  }
}
```

А.3 DSU – объединение по корням (structures/DSU.js)

```
union(a, b) {
  const ra = this.find(a);
  const rb = this.find(b);
  if (ra === rb) return false;    // уже в одной компоненте
  if (this.rank[ra] < this.rank[rb]) this.parent[ra] = rb;
  else if (this.rank[ra] > this.rank[rb]) this.parent[rb] = ra;
  else { this.parent[rb] = ra; this.rank[ra]++; }
  return true;
}
```

А.4 Алгоритм Дейкстры – бинарная куча и ленивое удаление (algorithms/dijkstra.js)

```
export function dijkstra(graph, startId) {
  const dist = new Map(), visited = new Set();
  for (const v of graph.getVertices()) dist.set(v.id, Infinity);
  dist.set(startId, 0);
  const pq = new PriorityQueue((a, b) => a[0] - b[0]);    // [dist, vertexId]
  pq.push([0, startId]);
  while (!pq.isEmpty()) {
    const [d, u] = pq.pop();
    if (visited.has(u)) continue;    // ленивое удаление
    if (d === Infinity) break;
    visited.add(u);
    for (const n of graph.getNeighbors(u)) {
      if (visited.has(n.to)) continue;
      const nd = d + n.weight;
      if (nd < dist.get(n.to)) {
        dist.set(n.to, nd);
        prev.set(n.to, u);
        pq.push([nd, n.to]);
      }
    }
  }
}
```

```

    }
  }

```

A.5 Алгоритм Краскала – сортировка и проверка цикла (algorithms/kruskal.js)

```

export function kruskal(graph) {
  const sorted = graph.getEdges().slice()
    .sort((a, b) => a.weight - b.weight || a.id - b.id);
  const dsu = new DSU(graph.vertexCount());
  const mstEdges = [];
  for (const e of sorted) {
    if (dsu.sameSet(idx(e.from), idx(e.to))) continue; // цикл
    dsu.union(idx(e.from), idx(e.to));
    mstEdges.push(e.id);
  }
}

```

Приложение Б

Результаты запуска тестов

В приложении приведён полный отчёт о запуске автоматизированных тестов в браузерном тест-раннере. Все 57 проверок проходят успешно.



Рисунок Б.1 – Результаты запуска автоматизированных тестов (браузерный тест-раннер)

Командная строка запуска тестов в Node.js: `node tests/run-all.mjs`. Результат: «Итого: 57 тестов, прошло 57, не прошло 0».